EasyNet Invocation Security Model

Reference Specification — Industrial-Grade Target Design

Silan Hu
National University of Singapore

Reference edition, April 14, 2026

Abstract

This document specifies the invocation security system for EasyNet at the rigor of RFC-class reference specifications. The body is *design-first*: it defines the target system as if from a blank slate, without reference to any current implementation. The central claim is that an EasyNet invocation is a cryptographic *statement* made by one or more principals about an intended operation, and the runtime's sole legitimate behavior is to evaluate that statement against a formally defined binding semantics, fail-closed. The design is grounded in three irreducible components: (i) per-layer cryptographic identity with explicit binding rules (§6.3), (ii) an SSH-shaped bootstrap separating single-use authorization tokens from durable identity keypairs (§7), and (iii) a mandatory signing-and-verification pipeline (§§8–9). It is aligned with and cites OpenSSH public-key authentication (RFC 4252), TLS 1.3 (RFC 8446), FIDO2/WebAuthn (W3C), SPIFFE/SPIRE (CNCF), NIST FIPS 186-5, FIPS 204, and the Dolev–Yao adversary model. Conformance of any particular implementation to this specification is addressed separately in the appendices.

Contents

1 Reading guide

This document uses the same three-tag discipline as `easynet_ontology.tex`, plus a fourth tag reserved for the conformance appendix.

[axiom] Either (i) a cited external standard (RFC, W3C, NIST, peer-reviewed paper) that this document adopts by reference, or (ii) a definition internal to this document that later derivations rest on. Axioms are load-bearing: changing one requires re-deriving every dependent statement.

[derived] A logical consequence of one or more axioms. Negotiable in review; the axioms are not.

[deferred] A known extension point, deliberately postponed to a later version of this document (§15).

non-conformance] Reserved for the conformance appendix (§A). A **[non-conformance]** tag identifies a deviation of a specific implementation from this specification. It never appears in the body.

Note

The body of this document (§§1–15) is design-first and self-contained: read in isolation, it is a complete reference specification suitable for clean-room implementation. The appendices (§A onward) analyze conformance of the present EasyNet codebase against this specification, for operational planning; they are not part of the design.

2 Scope

[axiom] This specification governs the authentication, integrity, and non-repudiation of *invocations* within an EasyNet runtime. An invocation is any control-plane or data-plane operation that names (i) a tenant, (ii) a resource or capability to be exercised, and (iii) zero or more principals claiming authorization for the operation.

[derived] The following operations are in scope: node registration, node heartbeat, ability invocation (synchronous, asynchronous, streaming), capability lifecycle management, mission dispatch, and any operation that returns an audit record.

[derived] The following are out of scope for this document:

- Confidentiality of invocation payloads. This is delegated to the underlying transport (TLS 1.3, RFC 8446 [7]).
- Anonymity and unlinkability of callers.
- Network-level denial-of-service defense.
- Billing, metering, and quota enforcement.
- Application-layer authorization (what a principal is *allowed* to invoke); this spec establishes the authenticated identity on which authorization may be built.

3 Adversary model

[axiom] The adversary is a strengthened Dolev–Yao adversary [1], consistent with the threat models of TLS 1.3 (RFC 8446 [7] §E), OpenSSH (RFC 4252 [3]), and SPIFFE [12]:

1. **Network control.** The adversary fully controls the network between any two parties: it may intercept, modify, replay, reorder, and inject arbitrary messages.
2. **Selective corruption.** The adversary may corrupt any finite set of non-target parties and learn their long-term keys.
3. **Cryptographic soundness.** The adversary cannot break Ed25519 signatures [9] or ECDSA-P256 (FIPS 186-5 [10]); cannot find SHA-256 preimages or collisions; cannot break the confidentiality, integrity, or authenticity of TLS 1.3.
4. **Filesystem boundaries.** Private keys stored with correct permissions on parties not under the adversary's control remain private.
5. **Public-key visibility.** The adversary may learn the full set of public keys the system has ever registered, including revoked ones.

[derived] Three consequences:

- Channel-level mTLS alone is insufficient: a corrupted party with a valid channel certificate is indistinguishable from an honest one at the channel layer.
- Bearer credentials (any secret whose possession is the credential) are strictly weaker than proof-of-possession signatures.
- Server-side custody of a principal's private key reduces that principal's identity to a proxy of the custodian.

4 Security goals

[axiom] This specification commits to the following six properties. Each is formalized to the level of RFC 4252 [3] §7 and WebAuthn [13] §13.

Security property

G1 (Authenticity). An invocation is accepted only if every identity it names is authenticated by a signature verifiable against a public key registered for that identity in state `Active`, under a private key the identity owner alone possesses.

Security property

G2 (Integrity). Any modification, by a party other than the signer, of any field the runtime uses to interpret the invocation (tenant, resource, function, arguments, nonce, timestamp, principal identifiers) invalidates the signature.

Security property

G3 (Non-repudiation). Every accepted invocation produces an audit record that independently verifies against the registered public keys. A signer cannot credibly deny having authorized an invocation whose signature verifies under its key.

Security property

G4 (Replay-resistance). An invocation captured off the wire cannot be successfully re-submitted, either within the nonce-reservation window (detected by nonce replay) or outside it (detected by timestamp skew).

Security property

G5 (Forward security of audit). Compromise of a private key at time t does not invalidate audit records produced at $t' < t$ while the key was in state `Active` and before revocation was applied.

Security property

G6 (Principle of least identity). An invocation asserts exactly the identities required to authorize the statement it makes. One identity's credentials cannot stand in for another's; delegation is an explicit, bounded, cryptographically evidenced operation (binding rule B4, §15).

[derived] G1–G6 together define what "authenticated" means in this specification. Confidentiality, anonymity, and unlinkability are explicitly not listed; their absence from this list is

intentional.

5 The invocation as a cryptographic statement

5.1 Definition

[axiom] An invocation I is a tuple

$$I = (T, P, N, A, f, \mathsf{args}, t_{\mathsf{iat}}, \mathsf{req\_id}, \mathsf{nonce})$$

where:

- T is the tenant under whose authority the invocation is made;
- P is the set of principals claiming authorization (possibly empty);
- N is the originating node (non-empty for any invocation that traverses an Axon runtime);
- A is the ability or capability being exercised, and f its specific function;
- args is the argument payload;
- t_{iat} is the Unix-millisecond timestamp at which I was issued;
- $\mathsf{req\_id}$ is a globally unique request identifier;
- nonce is at least 128 bits of cryptographic randomness.

[axiom] The canonical byte encoding of I, denoted $C(I)$, is a deterministic, injective serialization of the tuple under an unambiguous delimiter scheme.[1]

[derived] $C(I)$ includes $H(\mathsf{args})$ where H is SHA-256, not args itself; this keeps $C(I)$ bounded in size while preserving integrity of arbitrarily large payloads.

5.2 The authorization proposition

[axiom] Associated with I is the *authorization proposition* $\Phi(I)$:

> "Principal(s) P, acting under tenant T, transported by node N, authorize the execution of function f of ability A with arguments args at time t_{iat}."

The runtime's sole task, at the invocation boundary, is to decide whether $\Phi(I)$ is cryptographically established by the evidence accompanying I.

6 Identity model

6.1 Identity layers

[axiom] The specification recognizes three layers of cryptographic identity:

- **Node identity (K_N).** A keypair bound to a specific node (host, device, runtime instance). The private component is generated and retained on the node; the public component is registered with the runtime's identity service at pairing time.

[1] "Canonical" in the sense of RFC 8785 [8] and the SSH signing-blob format in RFC 4252 [3] §7: two parties computing $C(I)$ from the same I produce byte-identical results.

- **Principal identity (K_P).** A keypair bound to a logical principal (human user, service account, agent). Two registration modes are recognized, distinguished by who holds the private component:
  - *Strong principal identity.* The private component is generated and held exclusively by the principal's own trust boundary (e.g. a user's browser via WebCrypto [14]; a developer's local machine).
  - *Weak principal identity.* The private component is generated and held by an intermediary acting on the principal's behalf (e.g. a backend service-account key signing "on behalf of user U"). Weak identity is explicitly marked in the registration metadata and surfaced in the audit record.
- **Trust root identity (K_R).** The keypair or certificate authority that endorses the runtime's public identity, permitting clients to verify the runtime itself. This is the runtime's counterpart to SSH's `known_hosts`.

[axiom] All identities use the same underlying signature suite (see §8.3) so that evidence is uniform and pipelines are algorithm-agile.

6.2 Registration

[axiom] Every identity is associated with a *key record* of the form:

```
KeyRecord {
    subject:    { kind: Node | Principal | Root, id: string, tenant: string },
    algorithm:  Ed25519 | ECDSA-P256 | ...,
    version:    u32,              // monotonically increasing
    public_key: bytes,
    state:      Active | Rotated | Revoked,
    mode:       Strong | Weak,    // principals only; nodes are always Strong
    created_at: int64,
    rotated_at: int64,
    expires_at: int64,            // 0 = no expiry
    fingerprint: bytes            // SHA-256 of public_key
}
```

[derived] The `fingerprint` is redundant with `public_key` but provides a stable short identifier for audit records.

6.3 Binding semantics

[axiom] A signature σ by private key sk over canonical bytes $C(I)$, verifying against public key $K = \mathsf{pk}(sk)$, authorizes the low-level proposition:

> "The holder of the private key matching K authorized the byte sequence $C(I)$ at the time $C(I)$ claims."

Translating this into a statement about $\Phi(I)$ requires an explicit binding rule.

[axiom] This specification defines three binding rules. No other binding is permitted for v1.

Security property

B1 (Node origin). A signature σ_N under K_N (a node's registered key) over $C(I)$ authorizes the proposition "*invocation I originates from node N.*" It asserts nothing about any principal.

Security property

B2 (Principal authorization). A signature σ_P under K_P (a principal's registered key) over $C(I)$ authorizes the proposition "*principal P personally authorized invocation I.*" Strong B2 obtains when K_P is registered as `Strong`; weak B2 obtains when `Weak`. Both are B2; the distinction is evidentiary strength, not kind.

Security property

B3 (Conjunction). An invocation carrying both σ_N and σ_P, each verifying under its respective rule over the same $C(I)$, authorizes the conjunction $\Phi(I)$ in full: P personally authorized, N transported.

Invariant

(Binding non-substitutability.) A signature established under rule B_i shall not count as evidence for rule B_j, $j \neq i$. In particular, a node signature is not acceptable in lieu of a principal signature for invocations that name a principal, and a principal signature is not acceptable in lieu of a node signature for the node- origin claim.

[derived] When $\Phi(I)$ names k principals (e.g. a multi-party authorization), B2 is required k times, once per principal, each under the respective K_P. The general rule: the runtime accepts I only when $\Phi(I)$ is covered by the conjunction of all required bindings.

7 Bootstrap: separating authorization from identity

7.1 The problem bootstrap solves

[axiom] Identity registration is itself an operation that must be authorized, yet the identity being registered does not yet exist at authorization time. This chicken-and-egg problem is solved in OpenSSH [3] by separating two roles: a short-lived *bootstrap authorization* (e.g. the password accepted by `ssh-copy-id`), which authorizes exactly one transaction —the placement of a public key into `authorized_keys`—and a *durable identity* (the keypair) that authenticates every subsequent session.

7.2 Bootstrap tokens

[axiom] A *bootstrap token* is a high-entropy, server-issued, single-use credential. It satisfies:

- **High entropy.** At least 256 bits of cryptographic randomness, rendered in a URL-safe encoding.
- **Single-use.** The server atomically consumes it on validation; a second presentation fails.
- **Short-lived.** Validity is bounded by an explicit expiry (recommended: $\leq$ 15 minutes from issuance).
- **Narrowly scoped.** Authorizes one transaction of one kind: the registration of a single keypair under a single named identity.

[axiom] The bootstrap token authorizes the transaction "register public key K as the durable identity of subject S within tenant T." That is its entire authority. It is not an identity and is not accepted as evidence of any proposition about I after the registration transaction completes.

7.3 Durable keypair registration

[axiom] Durable keypair registration proceeds as follows. Let the prospective subject be S, the tenant T, and the bootstrap token τ:

1. S locally generates a fresh keypair (sk_S, K_S) using the operating system's cryptographic RNG (§10.1).
2. S computes a *possession proof* $\pi = \mathsf{Sign}(sk_S, C_{\mathsf{reg}})$ where C_{reg} is a canonical byte sequence committing to $(\tau, S, T, K_S, t_{\mathsf{iat}}, \mathsf{nonce})$.
3. S submits (τ, S, T, K_S, π) to the registration endpoint.
4. The server atomically (a) checks τ is unredeemed and unexpired, (b) marks τ redeemed, (c) verifies π under K_S, and (d) writes a `KeyRecord` binding K_S to S under T in state `Active`.

[derived] Step 2's possession proof prevents an adversary who intercepts τ from substituting its own public key: a forged $(\tau, S', T, K_{\mathsf{atk}}, \pi')$ still requires sk_{atk} to produce π', but binds the registration to K_{atk} rather than the legitimate K_S. The legitimate S, observing its token redeemed by another key, can raise an alert and request a fresh token.

7.4 The bootstrap invariant

Invariant

(Bootstrap separation.)

- A bootstrap token shall *never* appear on any wire carrying an operation other than identity registration, and *never* appear after the registration transaction has completed, whether successfully or not.
- A durable identity keypair's private component shall *never* be generated, stored, or transmitted by any party other than the principal that owns the identity. Servers, intermediaries, backups, and logs shall not contain it.

These two prohibitions are symmetric. Violating either collapses the separation.

[derived] Violating the first prohibition reduces identity to a bearer token, defeating G1 and G3. Violating the second turns the violating party into a custodian indistinguishable from the principal, defeating G3 (against the principal) and G6.

8 Signing pipeline

8.1 Canonicalization

[axiom] For every invocation I, the canonical byte sequence $C(I)$ is computed once at the signing boundary (before any hop) and carried unchanged through every intermediary until verification. Intermediaries MUST NOT re-canonicalize; they MUST NOT re-order or re-encode any field that contributes to $C(I)$.

[axiom] $C(I)$ is domain-separated by a protocol version string so that a signature valid under version v is not accepted under a different version.

8.2 Signature production

[derived] To produce evidence for a binding rule B over I:

- The signer computes $\sigma = \mathsf{Sign}(sk, C(I))$ using the private key corresponding to the identity the rule requires.
- The signer emits the tuple $(\sigma, K, \mathsf{kfp}(K), \mathsf{alg}(K))$ as companion metadata to I, where $\mathsf{kfp}(K)$ is the fingerprint and $\mathsf{alg}(K)$ is the algorithm identifier.
- Multiple binding rules contribute independent tuples; they are not combined, aggregated, or threshold-signed in v1.

8.3 Signature suite

[axiom] The v1 signature suite is:

Role	Algorithm	Reference
Primary	Ed25519	RFC 8032 [6], [9]
Alternate	ECDSA-P256 (SHA-256)	FIPS 186-5 [10]
Hash	SHA-256	FIPS 180-4
RNG	OS CSPRNG (`getrandom`, `BCryptGenRandom`)	NIST SP 800-90A

[deferred] Post-quantum signature suites (FIPS 204 ML-DSA [11], FIPS 205 SLH-DSA) are accommodated by the `algorithm` field in `KeyRecord` but not required in v1. See §15.

9 Verification pipeline

9.1 Mandatory verification

Invariant

(Mandatory verification, fail-closed.) The runtime shall evaluate every incoming invocation against the verification procedure of §9.2. An invocation for which the procedure does not return `Accept` shall be rejected. There shall be no configuration under which unverified invocations are silently accepted. Migration accommodations, if needed by an operator, shall take the form of *audit classes* emitted alongside rejection, never in place of it.

9.2 Verification procedure

[axiom] The runtime, for each incoming invocation I with accompanying signature tuples Σ, executes:

1. **Canonical reconstruction.** Reconstruct $C(I)$ from the invocation. If any required field of I is absent or unparseable, `Reject(MalformedCanonical)`.
2. **Payload integrity.** Verify that $H(\mathsf{args})$ equals the hash committed to in $C(I)$. Else `Reject(PayloadIntegrity)` (G2).
3. **Timestamp freshness.** Verify $|\mathsf{now} - t_{\mathsf{iat}}| \leq \Delta_{\mathsf{skew}}$. Else `Reject(StaleOrFuture)` (G4).
4. **Nonce reservation.** Atomically reserve nonce in the per-tenant nonce registry for a window of at least $2\Delta_{\mathsf{skew}}$. If already reserved, `Reject(Replay)` (G4).

5. **Binding coverage.** Compute the set of binding rules required to authorize $\Phi(I)$ (§6.3). For each rule, verify the corresponding tuple in Σ: the signature verifies under the claimed public key; the public key resolves in the key registry to a record of state `Active`, subject matching the rule, tenant matching T, algorithm matching the algorithm indicator; and the expiry, if any, is not past. Any mismatch yields `Reject(BindingUnsatisfied)` (G1).
6. **Non-substitutability.** Verify that no single signature is being applied to more than one binding rule (invariant of §6.3). Else `Reject(BindingCollision)`.
7. **Audit commit.** Emit an audit record containing req_id, T, P, N, A, f, kfp$(\cdot)$ for each verifying key, and each signature σ. This record is the non-repudiation artifact (G3, G5).
8. `Accept`. Dispatch I to execution.

9.3 No opt-in mode

[derived] Verification is not gated on the presence of any opt-in signal in the invocation (no "if-metadata-present-then-verify" shortcut). Invocations lacking signature tuples are not "legacy clients" to the runtime; they are rejected. Operators preparing for deployment are expected to upgrade clients before enabling the runtime, not to leave enforcement conditional on the client's cooperation.

10 Key lifecycle

10.1 Generation

[axiom] Private keys are generated on the party that owns them, using the host operating system's cryptographic RNG:

- Node keys: generated on the node at first boot, or at any subsequent rotation.
- Strong principal keys: generated in the principal's ultimate trust boundary (browser WebCrypto [14], CLI-local, hardware token).
- Weak principal keys (service accounts): generated on the hosting service at bootstrap, stored per the service's own secrets-management policy.
- Trust root keys: generated by the operator at runtime initialization; rotation is a privileged operator action.

[axiom] Deterministic key derivation from public or low-entropy inputs is prohibited. A key derived from inputs knowable to the adversary is not a key.

10.2 Storage

[axiom] Private keys at rest are protected by, minimally, host operating-system access control (POSIX mode `0600` or the equivalent ACL). Implementations SHOULD additionally support: passphrase encryption (scrypt [5] key derivation, authenticated symmetric encryption), OS key storage (macOS Keychain, Linux Secret Service, Windows DPAPI), hardware-backed storage (TPM, Secure Enclave, YubiKey, HSM).

[derived] The signing interface is parameterized over a *signer* abstraction so that storage backends can be added without changes to canonicalization or verification.

10.3 Rotation

[axiom] Rotation of an identity's keypair proceeds as follows:

1. The identity generates a fresh keypair (sk', K') per §10.1.
2. The identity produces a *rotation proof* $\pi_{\mathsf{rot}} = \mathsf{Sign}(sk, C_{\mathsf{rot}})$ under the current active key sk, where C_{rot} canonically commits to $(S, T, K', K, t_{\mathsf{iat}}, \mathsf{nonce})$.
3. The identity submits $(S, T, K', \pi_{\mathsf{rot}})$ to the runtime's identity service.
4. The runtime verifies π_{rot} under K, atomically transitions the prior `KeyRecord` to state `Rotated`, inserts a new record with state `Active` and version incremented, and records the rotation in audit.

[derived] The rotation proof binds the new key to the prior key's authority: only the current legitimate holder can authorize rotation. An adversary who obtains sk can rotate, but this is detectable (the legitimate holder observes a rotation it did not initiate) and recoverable (revoke both; re-bootstrap).

10.4 Revocation

[axiom] Revocation transitions a `KeyRecord` from any state to `Revoked`. Signatures produced under a revoked key are rejected at step 5 of the verification procedure from the moment revocation is visible to the runtime.

[derived] Revocation is authorized by one of: (i) the identity itself, via a signature by the current key (self-revocation); (ii) the trust root, via a signature by K_R (administrative revocation); (iii) out-of-band operator action (break-glass), which MUST emit a distinguished audit class.

[derived] Revocation is eventually consistent across federated runtimes; cross-runtime propagation is **[deferred]** (§15).

10.5 Expiry

[derived] A `KeyRecord` with `expires_at` > 0 and `expires_at` $\leq$ `now` is treated as if revoked at verification time, without requiring an explicit revocation operation. Expiry policy (default per-tenant lifetimes, renewal grace periods) is **[deferred]**.

11 Trust roots and host verification

[axiom] The runtime exposes its own public identity K_R through a trust-root service. Clients pin K_R at first contact, analogous to SSH's `known_hosts` (RFC 4251 [2] §4.1) or WebPKI certificate pinning (RFC 7469 [4]).

[derived] Pinning mitigates the Dolev–Yao adversary's ability to impersonate the runtime: after first contact, a client rejects a runtime presenting a different K_R unless a trust-root rotation proof is supplied.

[axiom] Trust-root rotation is itself an operation subject to this specification: a rotation proof signed by the outgoing K_R over the incoming K'_R, countersigned as operator policy requires.

[deferred] Multi-root endorsement ("the operator delegates trust to a certificate authority"), cross-runtime trust in federation, and the ceremony for the very first K_R (bootstrap of the root) are out of v1 scope.

12 Protocol surface

This section states the minimum protocol surface the specification requires. It is intentionally algorithm-agile and wire-format-agnostic beyond the canonicalization requirements of §8.

12.1 Invocation envelope

[axiom] Every invocation carries an *envelope* with, minimally, the fields of tuple I (§5.1): tenant, principals, node, resource, function, arguments, timestamp, request id, nonce. Absence of any mandatory field at verification time yields `Reject(MalformedCanonical)`.

12.2 Signature metadata

[axiom] Signature tuples travel alongside the envelope, in metadata clearly separable from the envelope (so the envelope's canonical form is not perturbed). For each binding rule B claimed, metadata includes: the signature σ, the public key K, the key fingerprint $\mathsf{kfp}(K)$, the algorithm identifier $\mathsf{alg}(K)$, and the binding rule identifier $B \in \{B1, B2, B3, \ldots\}$.

12.3 Identity-service RPCs

[axiom] The runtime exposes at least the following identity-service operations:

- `RegisterKey`: the bootstrap registration of §7.3.
- `RotateKey`: the rotation of §10.3.
- `RevokeKey`: the revocation of §10.4.
- `GetTrustRoot`: the public identity of the runtime.
- `RotateTrustRoot`: administrative trust-root rotation.
- `VerifySignature`: a standalone verification endpoint for offline audit verification.

[axiom] Every identity-service RPC is itself an invocation subject to this specification. Bootstrap registration is the single base case: it is authorized by the bootstrap token rather than a signature, exactly because it creates the signing capability that all subsequent RPCs will require.

13 Audit

[axiom] Every `Accept` outcome of the verification procedure produces an audit record containing: req_id, tenant, principal set with per-principal kfp and B2 mode (strong / weak), node with its kfp, resource and function, argument hash, timestamp, binding rules satisfied, and each signature σ.

[axiom] Every `Reject` outcome produces an audit record containing the same identifying fields (to the extent they parsed successfully) plus the `Reject` reason code.

[derived] The record's inclusion of raw signatures and public-key fingerprints is what establishes G3 (non-repudiation) and G5 (forward security): any later verifier with access to the historical key registry can independently re-verify accepted records.

[axiom] Audit records are append-only; amendments take the form of new records, not mutations of old ones. Retention policy, storage location, and access control are out of v1 scope.

14 Construction of security properties

[derived] This section traces how G1–G6 follow from the mechanisms above. The traces are informal but machine-checkable against the preceding sections.

Goal	Construction
G1	Step 5 of §9.2 rejects invocations whose identity claims lack a signature that verifies under a registered `Active` key. Under §3's Ed25519 assumption, only the private-key holder can produce such a signature. §10.1 requires private keys be generated by the holder and forbids derivation from public inputs; §7.4 forbids custody transfer.
G2	Step 2 recomputes $H(\mathsf{args})$ from the on-wire payload and requires equality with the value inside $C(I)$. Canonicalization covers every other routing-relevant field (§5.1). Modification by a third party yields an invalid signature under G1's construction.
G3	Step 7 commits the signatures, public-key fingerprints, and invocation fields to audit. A later verifier, with access to the public-key registry at the time of acceptance, can re-run steps 5–6 offline against the audited record. Strong vs. weak B2 is recorded to make the scope of the non-repudiation claim explicit.
G4	Steps 3 and 4 combine: within one skew window, the nonce registry rejects replays; outside it, timestamp freshness rejects captured-and-resubmitted invocations. The window is per tenant to prevent cross-tenant nonce collision-space exhaustion.
G5	Audit records (§13) include the key fingerprint at acceptance time, not a dynamic reference to the key record. Later revocation updates the registry (rejecting new invocations) but leaves historical records semantically unchanged. Revocation under §10.4 records its own audit entry, so the transition t'_{revoked} is itself non-repudiable.
G6	§6.3's invariant forbids binding substitution. §6.1's distinction between strong and weak principal identity prevents a weak signature from silently standing in for a strong one: the audit record surfaces the mode, and authorization policy above this layer may require strong B2 for sensitive operations. Delegation (B4) is explicitly **[deferred]**, not tacitly absorbed into B2.

15 Open extensions

These extensions are compatible with the v1 design and are deliberately deferred.

1. **Hardware- and OS-backed key storage.** Secure Enclave, TPM, YubiKey, HSM, macOS Keychain, Windows DPAPI, Linux Secret Service. Accommodated by the signer abstraction of §10.2; no protocol change.

2. **Post-quantum signatures.** NIST FIPS 204 ML-DSA and FIPS 205 SLH-DSA [11]. Accommodated by `algorithm` in `KeyRecord`; a PQC-only deployment or hybrid deployment (classical + PQC) is additive.

3. **Binding rule B4 (delegation).** A principal A authorizing a principal B to act on its behalf in a bounded scope and for a bounded time. Candidate designs include macaroons [15], capability URLs [16], biscuits, and SPIFFE federation tokens. v1 explicitly forbids delegation to avoid committing to a design prematurely.

4. **Per-tenant policy.** Skew window, nonce TTL, required-mode for principals (strong-B2-only), accepted algorithms, key expiry policy. v1 uses runtime-global defaults.

5. **Cross-runtime federation identity.** Revocation propagation, cross-tenant naming, federation-level trust roots. SPIFFE [12] trust-bundle federation is the reference design.

6. **Workload identity.** Non-interactive pairing for automation (CI runners, sidecars, initContainers) along the lines of SPIFFE `WorkloadAPI` and Kubernetes projected service account tokens.

7. **Anonymous credentials and unlinkability.** Group-signature-based and attribute-based constructions for cases where the principal wishes not to be linked across invocations. Orthogonal to v1's non-repudiation goal, hence **[deferred]**.

8. **Threshold signatures.** A binding rule that requires k-of-n key holders to jointly sign. Useful for high-stakes operations (trust-root rotation, cross-tenant delegation). Out of v1 because the additional protocol complexity is not justified by v1 use cases.

9. **Formal verification.** The specification is amenable to mechanized proof in Tamarin [17] or ProVerif. A formal model is out of v1 but the canonicalization and binding rules are structured to make formalization tractable.

16 Conclusion

The load-bearing content of this specification is three items.

Mandatory verification, fail-closed (§9): every invocation is evaluated against a fixed procedure; invocations that do not satisfy it are rejected; there is no opt-in shortcut.

Binding semantics (§6.3): node-origin and principal-authorization are distinct claims, each requiring its own signature; neither substitutes for the other; conjunction is the only way to claim both.

Bootstrap separation (§7): single-use authorization tokens and durable identity keypairs play non- interchangeable roles; the private component of a durable keypair lives only on the party that owns the identity.

Everything else—the signing pipeline, the verification procedure, the key lifecycle, the audit record—follows mechanically from these three. An implementation that satisfies all three satisfies G1 through G6 under the adversary model of §3. An implementation that relaxes any of them does not.

Extensions enumerated in §15 are additive: they augment this specification without revisiting its axioms. The specification is designed to be conformant-once, extended-many-times.

References

[1] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[2] T. Ylonen and C. Lonvick (Eds.). The Secure Shell (SSH) Protocol Architecture. RFC 4251, IETF, 2006.

[3] T. Ylonen and C. Lonvick (Eds.). The Secure Shell (SSH) Authentication Protocol. RFC 4252, IETF, 2006.

[4] C. Evans, C. Palmer, and R. Sleevi. Public Key Pinning Extension for HTTP. RFC 7469, IETF, 2015.

[5] C. Percival and S. Josefsson. The scrypt Password-Based Key Derivation Function. RFC 7914, IETF, 2016.

[6] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IETF, 2017.

[7] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, IETF, 2018.

[8] A. Rundgren, B. Jordan, and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, IETF, 2020.

[9] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[10] National Institute of Standards and Technology. Digital Signature Standard (DSS). FIPS PUB 186-5, 2023.

[11] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard. FIPS PUB 204 (ML-DSA), 2024.

[12] SPIFFE Authors. The SPIFFE Identity and Verifiable Identity Document (SPIFFE ID, SVID). SPIFFE Specifications v1.0+, `https://spiffe.io/`.

[13] D. Balfanz et al. Web Authentication: An API for accessing Public Key Credentials, Level 2. W3C Recommendation, 2021.

[14] M. Watson (Ed.). Web Cryptography API. W3C Recommendation, 2017.

[15] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS*, 2014.

[16] M. S. Miller. Robust composition: Towards a unified approach to access control and concurrency control. PhD thesis, Johns Hopkins University, 2006.

[17] S. Meier, B. Schmidt, C. Cremers, and D. Basin. The Tamarin Prover for the symbolic analysis of security protocols. In *CAV*, 2013.

A Conformance of the present codebase

This appendix is operational, not normative. It identifies the deviations of the current EasyNet codebase (as observed at the time of this edition) from the specification above. Each deviation is tagged **[non-conformance]**, cited to file path and line number, and mapped to the section of the specification it violates.

Note

The body of this document would remain valid, as a reference specification, if this appendix were deleted. The appendix exists to direct implementation work, not to influence the design.

A.1 Summary

[non-conformance] The present runtime accepts invocations that lack any identity evidence, by virtue of an opt-in branch in the verifier (violates §9). The three identity layers (§6) exist as separate artifacts—client-side bearer token, backend-side JWT, runtime-side keypair infrastructure— none of which is transported end-to-end to the verification boundary. The runtime executes in a state operationally indistinguishable from "unauthenticated" while appearing, to a surface audit, to be authenticated. The specific loci are enumerated below.

A.2 Non-conformance catalog

Each entry names the deviation, the file locations at which it manifests, and the section of the specification it violates. File paths are given as verbatim identifiers to be resolvable without ambiguity; they are not links.

C1 — opt-in verification **Violates:** §9. The verifier returns `Ok(None)` when the expected `easynet.*` metadata is absent, short-circuiting the verification procedure and allowing the invocation to dispatch.
Location: `core/runtime-rs/src/runtime/security.rs`, in `verify_easynet_invocation_metadata` (the early return when `easynet.resource_uri` is absent).

C2 — tenant-only gate **Violates:** §9. The sole authentication gate invoked from invoke handlers checks envelope tenant presence only; no binding evaluation is performed.
Location: `core/runtime-rs/src/runtime/mod.rs`, in `require_envelope_tenant`.

C3 — backend interface has no principal slot **Violates:** §6. The backend's `CallMcpTool` (and `CallMcpToolStream`) interface has no parameter for a principal identifier; user identity captured from the JWT cannot reach the runtime.
Location: `backend/internal/axon/client.go`, interface definition.

C4 — principal dropped at Axon call site **Violates:** §6. The backend ability-invoke logic captures `uid` from the request context, then calls the Axon client without transporting it.
Location: `backend/internal/logic/ability/invokeAbilityLogic.go`.

C5 — `credential_token` conflates roles **Violates:** §7. A single field `credential_token` is used both as bootstrap proof (issued and redeemed at pairing) and as durable identity (stored at rest and replayed at every startup), collapsing the distinction the specification requires.
Locations: `EasyNet-Cli/src/shared/config.rs` (the `credential_token` field of `Credentials`); `backend/internal/logic/device/validatePairingLogic.go` (issuance of plaintext token); `EasyNet-Cli/src/cli/start.rs` (replay on every startup).

C6 — deterministic-derivation "keys" **Violates:** §10. The client SDK derives an Ed25519 seed deterministically from `tenant_id` and `subject_id` using a public salt, producing a private key recoverable by any party with access to the SDK source.
Location: `core/runtime-rs/client-sdk/src/domain/easynet/semantic.rs`, in `derive_subject_auth`.

C7 — envelopes omit `easynet` metadata **Violates:** §§8, 9. CLI client envelopes sent to the runtime have the `easynet` metadata field unset; no signature tuple is produced at the signing boundary.
Location: `core/runtime-rs/client-sdk/src/infra/client.rs`, envelope construction in the `heartbeat` and `invoke` paths, where the `easynet` field is left `None`.

C8 — CLI call paths never sign **Violates:** §§8, 9. Heartbeat and ability-invoke call paths in the CLI do not construct or attach signature metadata.
Locations: `EasyNet-Cli/src/cli/heartbeat.rs`; `EasyNet-Cli/src/cli/invoke.rs`.

A.3 Resolution phasing

[derived] The observed deviations can be resolved without a specification change. Suggested phasing:

- **Phase R0 (code parity).** Resolve C6, C7, C8 on the signing side and C3, C4 on the transport side. The runtime still accepts unsigned invocations; audit distinguishes signed from unsigned classes. No invocation is rejected in this phase; the goal is to make every real caller capable of producing signatures.
- **Phase R1 (enforcement).** Resolve C1, C2. The verifier becomes mandatory; unsigned invocations are rejected. Audit data from R0 is reviewed to confirm the unsigned class is empty before this phase begins.
- **Phase R2 (cleanup).** Resolve C5 by removing `credential_token` from persisted credentials and from the backend schema; the device's durable identity is the keypair alone.

[derived] Phasing is an operational choice; the specification itself requires only the end state (all of C1–C8 resolved). Permanent retention of the R0 state is not a valid long-term configuration.

The body of this document (§§1–15) is a reference specification, design-first and implementation-agnostic. The appendices analyze a specific codebase's conformance to it. The two concerns are kept separate so the specification remains useful to future clean-room implementations without inheriting the contingencies of the present codebase.